

Implementation of Lock-Based Savings Account

Objective

The goal of this assignment is to design and implement a lock-based savings account system that supports:

- **Deposit:** Adding funds to the account.
- **Ordinary Withdrawal:** Removing funds if the balance is sufficient.
- **Preferred Withdrawal (Extra Credit):** Ensuring withdrawals marked as preferred are prioritized over ordinary withdrawals.

Program Design

Classes and Their Attributes

- **Logger Class:** Handles logging of events for debugging and output purposes.
 - **DEBUG(...):** Logs debugging information to a file.
 - **OUTPUT(...):** Logs detailed operations with timestamps, thread IDs, and account balances.
- **Account Class:** Manages individual account operations and synchronization.
 - **Attributes:**
 - **balance:** Current balance of the account.
 - **mtx:** Mutex to ensure mutual exclusion for account operations.
 - **cv:** Condition variable for notifying threads waiting for withdrawals.
 - **requestQueue:** Queue to manage pending withdrawal requests.
 - **Constructor:** Initializes the account with a starting balance.
- **Request Class:** Represents a withdrawal request.
 - **Attributes:**
 - **is_preferred:** Indicates if the withdrawal is prioritized.
 - **amount:** Amount to be withdrawn.
 - **cv:** Pointer to the condition variable for this request.

Helper Functions

- **getSysTime:** Converts a time point into a human-readable string for logging timestamps.
- **processQueue:** Processes pending withdrawal requests based on the available balance and priority.

Concurrency Mechanisms

- **Mutex:** Ensures mutual exclusion for all account operations.
- **Condition Variables:** Blocks threads waiting for conditions (e.g., sufficient balance) to be met.

Program Flow

1. **Initialization:** Reads input parameters and sets up accounts and threads.
2. **Thread Function:** Each thread performs a sequence of deposit or withdrawal operations, based on randomized inputs.
3. **Logging:** All operations are logged with details of thread IDs, actions, timestamps, and account states.
4. **Completion:** Ensures all threads complete their operations and outputs final statistics.

Correctness Criteria

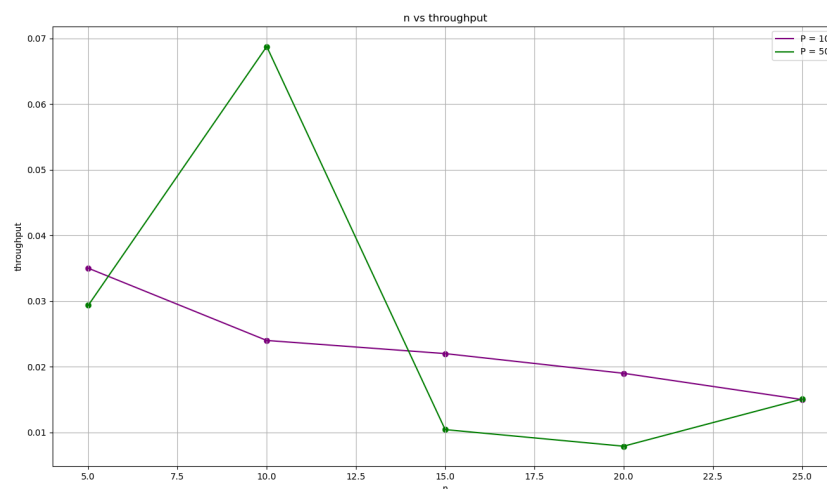
The implementation satisfies the following criteria:

1. **Mutual Exclusion:** Ensures no two threads perform conflicting operations on the same account simultaneously.
2. **Starvation Freedom:** All requests (ordinary or preferred) are eventually processed.
3. **Priority Handling:** Preferred withdrawals are processed before ordinary ones.

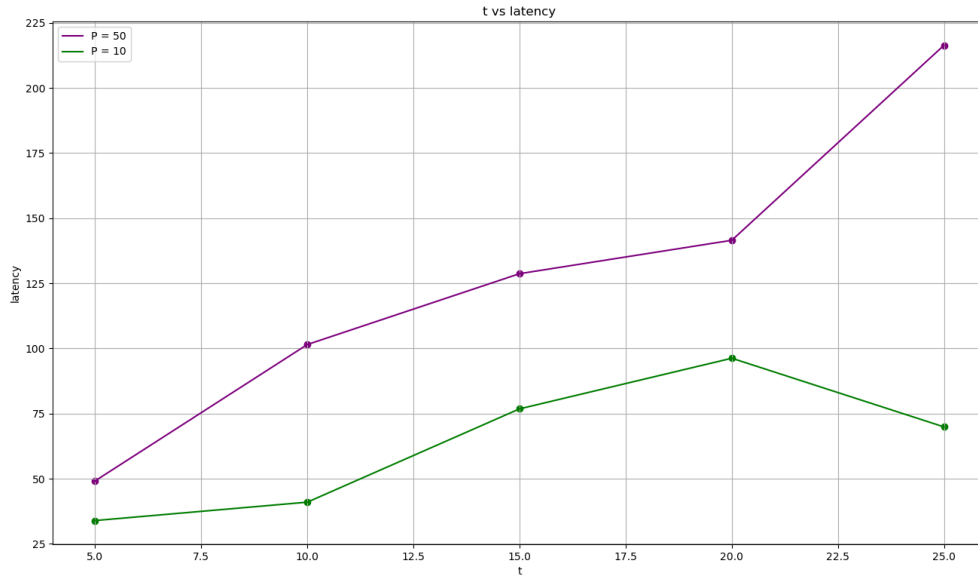
Results and Observations

Experiment 1: Throughput vs. Number of Threads

- **Setup:** Number of threads varied from 5 to 25, with and $\alpha = 1.5$.
- **Observation:** Throughput increases with threads but saturates due to synchronization overhead.



- **Experiment 2: Latency vs. Number of Operations**
- **Setup:** Number of operations per thread varied, with , , and $\alpha = 1.5$.
- **Observation:** Latency increases the number of operations.



Experiment 3: Time vs. Number of Accounts

- **Setup:** Number of accounts varied from 10 to 50, with , $\alpha = 1.5$, and (t = 20).
- **Observation:** Processing time increases as the number of accounts grows due to increased contention.

